

Q.4: FCFS Process Scheduling

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[20], arrival[20], burst[20];
    int completion[20], turnaround[20], waiting[20];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "\nProcess " << pid[i] << ": \n";
        cout << " Enter arrival time: ";
        cin >> arrival[i];
        cout << " Enter burst time: ";
        cin >> burst[i];
    }

    // Sort all processes by arrival time (FCFS order) using simple bubble sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arrival[j] > arrival[j + 1]) {
                // swap arrival
                int temp = arrival[j];
                arrival[j] = arrival[j + 1];
                arrival[j + 1] = temp;

                // swap burst
                temp = burst[j];
                burst[j] = burst[j + 1];
                burst[j + 1] = temp;

                // swap pid
                temp = pid[j];
                pid[j] = pid[j + 1];
                pid[j + 1] = temp;
            }
        }
    }

    int currentTime = 0;
    int totalTurnaround = 0, totalWaiting = 0;

    for (int i = 0; i < n; i++) {
        // If CPU is idle before this process arrives, move time forward
        if (currentTime < arrival[i])
            currentTime = arrival[i];

        currentTime = currentTime + burst[i];
    }
}
```

```

        completion[i] = currentTime;

        turnaround[i] = completion[i] - arrival[i];
        waiting[i] = turnaround[i] - burst[i];

        totalTurnaround = totalTurnaround + turnaround[i];
        totalWaiting = totalWaiting + waiting[i];
    }
    cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
    for (int i = 0; i < n; i++) {
        cout << "P" << pid[i] << "\t"
            << arrival[i] << "\t"
            << burst[i] << "\t"
            << completion[i] << "\t\t"
            << turnaround[i] << "\t\t"
            << waiting[i] << endl;
    }
    float avgTurnaround = (float)totalTurnaround / n;
    float avgWaiting = (float)totalWaiting / n;

    cout << "\nAverage Turnaround Time = " << avgTurnaround << endl;
    cout << "Average Waiting Time = " << avgWaiting << endl;
    return 0;
}

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

cd "/Users/pratikthapamagar/OS_lab_sem4/" && g++ lab_4.cpp -o lab_4 && "/Users/pratikthapamagar/OS_l
ab_sem4/"lab_4
● pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 % cd "/Users/pratikthapamagar/OS_lab_sem4/" && g++
lab_4.cpp -o lab_4 && "/Users/pratikthapamagar/OS_lab_sem4/"lab_4
Enter number of processes: 4

Process 1:
Enter arrival time: 1
Enter burst time: 20

Process 2:
Enter arrival time: 3
Enter burst time: 5

Process 3:
Enter arrival time: 0
Enter burst time: 15

Process 4:
Enter arrival time: 2
Enter burst time: 10

Process Arrival Burst Completion Turnaround Waiting
P3 0 15 15 15 0
P1 1 20 35 34 14
P4 2 10 45 43 33
P2 3 5 50 47 42

Average Turnaround Time = 34.75
Average Waiting Time = 22.25
○ pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 %

```

Q.5: SJF Process Scheduling

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[20], arrival[20], burst[20];
    int completion[20], turnaround[20], waiting[20];
    bool done[20];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "\nProcess " << pid[i] << ":\n";
        cout << " Enter arrival time: ";
        cin >> arrival[i];
        cout << " Enter burst time: ";
        cin >> burst[i];
        done[i] = false;
    }

    int currentTime = 0;
    int completed = 0;
    int totalTurnaround = 0, totalWaiting = 0;

    while (completed < n) {
        // Find the process with the shortest burst time among those
        // that have arrived by currentTime and are not yet completed
        int idx = -1;
        int minBurst = 9999;

        for (int i = 0; i < n; i++) {
            if (!done[i] && arrival[i] <= currentTime) {
                if (burst[i] < minBurst) {
                    minBurst = burst[i];
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            // No process has arrived yet, move time forward
            currentTime++;
            continue;
        }

        // Run the selected process to completion
        currentTime = currentTime + burst[idx];
        completion[idx] = currentTime;
        turnaround[idx] = completion[idx] - arrival[idx];
        waiting[idx] = turnaround[idx] - burst[idx];
    }
}
```

```

        totalTurnaround = totalTurnaround + turnaround[idx];
        totalWaiting = totalWaiting + waiting[idx];

        done[idx] = true;
        completed++;
    }
    cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
    for (int i = 0; i < n; i++) {
        cout << "P" << pid[i] << "\t"
            << arrival[i] << "\t"
            << burst[i] << "\t"
            << completion[i] << "\t\t"
            << turnaround[i] << "\t\t"
            << waiting[i] << endl;
    }
    float avgTurnaround = (float)totalTurnaround / n;
    float avgWaiting = (float)totalWaiting / n;

    cout << "\nAverage Turnaround Time = " << avgTurnaround << endl;
    cout << "Average Waiting Time = " << avgWaiting << endl;

    return 0;
}

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 % g++ lab_5.cpp -o lab_5
● pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 % cd "/Users/pratikthapamagar/OS_lab_sem4/" && g++
lab_5.cpp -o lab_5 && "/Users/pratikthapamagar/OS_lab_sem4/"lab_5
Enter number of processes: 4

Process 1:
Enter arrival time: 1
Enter burst time: 20

Process 2:
Enter arrival time: 3
Enter burst time: 5

Process 3:
Enter arrival time: 0
Enter burst time: 15

Process 4:
Enter arrival time: 2
Enter burst time: 10

Process Arrival Burst Completion Turnaround Waiting
P1 1 20 50 49 29
P2 3 5 20 17 12
P3 0 15 15 15 0
P4 2 10 30 28 18

Average Turnaround Time = 27.25
Average Waiting Time = 14.75
○ pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 %

```

Q.6: Round Robin Process Scheduling

```
#include <iostream>
using namespace std;

int main()
{
    int n, timeQuantum;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[20], arrival[20], burst[20], remaining[20];
    int completion[20], turnaround[20], waiting[20];

    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "\nProcess " << pid[i] << ":\n";
        cout << " Enter arrival time: ";
        cin >> arrival[i];
        cout << " Enter burst time: ";
        cin >> burst[i];
        remaining[i] = burst[i];
    }

    cout << "\nEnter time quantum: ";
    cin >> timeQuantum;

    int currentTime = 0;
    int completed = 0;

    // Simple queue implemented using an array
    int queue[1000];
    int front = 0, rear = 0;
    bool inQueue[20];

    for (int i = 0; i < n; i++)
        inQueue[i] = false;

    // Add any process that has already arrived at time 0
    for (int i = 0; i < n; i++) {
        if (arrival[i] <= currentTime && !inQueue[i]) {
            queue[rear++] = i;
            inQueue[i] = true;
        }
    }

    while (completed < n) {
        if (front == rear) {
            // Queue is empty, move time forward to next arrival
            currentTime++;
            for (int i = 0; i < n; i++) {
                if (arrival[i] <= currentTime && !inQueue[i]) {
                    queue[rear++] = i;
                }
            }
        }
        // Process at front of queue
        int i = queue[front];
        remaining[i] -= timeQuantum;
        if (remaining[i] <= 0) {
            completion[i] = currentTime;
            completed++;
        }
        front++;
    }
}
```

```

        inQueue[i] = true;
    }
}
continue;
}

int cur = queue[front++];

if (remaining[cur] > timeQuantum) {
    currentTime = currentTime + timeQuantum;
    remaining[cur] = remaining[cur] - timeQuantum;
} else {
    currentTime = currentTime + remaining[cur];
    remaining[cur] = 0;
}

// After running this process, add any new arrivals (in order)
for (int i = 0; i < n; i++) {
    if (arrival[i] <= currentTime && !inQueue[i]) {
        queue[rear++] = i;
        inQueue[i] = true;
    }
}

if (remaining[cur] == 0) {
    completion[cur] = currentTime;
    turnaround[cur] = completion[cur] - arrival[cur];
    waiting[cur] = turnaround[cur] - burst[cur];
    completed++;
} else {
    // Not finished, put it back at the end of the queue
    queue[rear++] = cur;
}
}

int totalTurnaround = 0, totalWaiting = 0;
for (int i = 0; i < n; i++) {
    totalTurnaround = totalTurnaround + turnaround[i];
    totalWaiting = totalWaiting + waiting[i];
}

cout << "\nProcess\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\n";
for (int i = 0; i < n; i++) {
    cout << "P" << pid[i] << "\t"
        << arrival[i] << "\t"
        << burst[i] << "\t"
        << completion[i] << "\t\t"
        << turnaround[i] << "\t\t"
        << waiting[i] << endl;
}

float avgTurnaround = (float)totalTurnaround / n;
float avgWaiting = (float)totalWaiting / n;

```

```

    cout << "\nAverage Turnaround Time = " << avgTurnaround << endl;
    cout << "Average Waiting Time = " << avgWaiting << endl;

    return 0;
}

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 % cd "/Users/pratikthapamagar/OS_lab_sem4/" && g++
  lab_6.cpp -o lab_6 && "/Users/pratikthapamagar/OS_lab_sem4/"lab_6
Enter number of processes: 4

Process 1:
  Enter arrival time: 0
  Enter burst time: 20

Process 2:
  Enter arrival time: 7
  Enter burst time: 25

Process 3:
  Enter arrival time: 9
  Enter burst time: 10

Process 4:
  Enter arrival time: 15
  Enter burst time: 15

Enter time quantum: 6

Process Arrival Burst Completion Turnaround Waiting
P1      0      20      48          48          28
P2      7      25      70          63          38
P3      9      10      46          37          27
P4     15      15      63          48          33

Average Turnaround Time = 49
Average Waiting Time = 31.5
○ pratikthapamagar@Pratiks-MacBook-Pro OS_lab_sem4 %

```